

AI Repository Architect

Automated Repository Documentation Report

Repository ID: 7fab68b7-f139-48a7-9509-d4ee960cb41f

Generated using AI-powered code analysis

Project Overview

This project is a fantasy RPG duel narrative generator that leverages a locally-hosted open-source Language Model (LLM) to provide dynamic and engaging descriptions for each move in a chess game. The application is designed to enhance the gaming experience by integrating rich storytelling and visual effects based on the chess moves.

Architecture

The architecture of this project is centered around a single core function that interacts with a locally-hosted LLM to generate narrations and effects for chess moves. The system includes fallback mechanisms to ensure that the game remains functional even if the LLM is unavailable or returns invalid data.

Key Components:

- **LLM Interaction**: Communication with the locally-hosted LLM to generate narrations and effects.
- **Fallback Mechanism**: Provides default narrations and effects when the LLM interaction fails.
- **Prompt Construction**: Builds the input prompt for the LLM based on the chess move information.
- **Data Parsing**: Parses the JSON response from the LLM to extract the narration and effect.
- **Error Handling**: Manages exceptions during LLM communication and ensures the application continues to run smoothly.

Folder Structure

The repository is organized as follows:

...

repo/

■

■■■ main.py # Main script containing the core functions

■■■ utils.py # Utility functions and constants

'''

Key Components

Constants

- ``OLLAMA_URL``: URL where the locally-hosted LLM is accessible.
- ``MODEL_NAME``: Name of the LLM model being used.
- ``VALID_EFFECTS``: List of valid visual effects that can be returned by the LLM.
- ``PIECE_NAMES``: Mapping of chess pieces to their fantasy RPG equivalents.
- ``FALLBACK_EFFECTS``: Dictionary mapping categories of moves to lists of fallback effects.

Utility Functions

- ``_category(move_info)``: Determines the category of a move (e.g., checkmate, capture).
- ``_fallback_narration(move_info)``: Generates a default narration for a move.
- ``_fallback_effect(move_info)``: Selects a random fallback effect for a move.

Main Functions

``get_narration(move_info, timeout=8)``

- **Description**: This function generates a narration and effect for a given chess move by interacting with the LLM. It handles exceptions and uses fallback narrations and effects if necessary.
- **Parameters**:
 - ``move_info``: A dictionary containing details about the chess move.
 - ``timeout``: Maximum time to wait for a response from the LLM (default is 8 seconds).
- **Returns**: A dictionary containing the narration, effect, source of the data, and optionally an error message.

``build_prompt(move_info)``

- **Description**: Constructs a prompt for the LLM based on the chess move information.
- **Parameters**:
 - `move_info`: A dictionary containing details about the chess move.
- **Returns**: A string formatted as a JSON object, which serves as the prompt for the LLM.

API Endpoints

This project does not expose any external API endpoints. It is designed to be integrated into a larger application where the `get_narration` function can be called with move information to receive narrations and effects.

Data Flow

1. **Chess Move Input**: The application receives information about a chess move.
2. **Prompt Construction**: The `build_prompt` function constructs a prompt based on the move information.
3. **LLM Interaction**: The `get_narration` function sends the prompt to the LLM and waits for a response.
4. **Response Parsing**: The JSON response from the LLM is parsed to extract the narration and effect.
5. **Validation**: The extracted narration and effect are validated against predefined criteria.
6. **Fallback Handling**: If the LLM response is invalid or an exception occurs, fallback narrations and effects are used.
7. **Output**: The function returns a dictionary containing the narration, effect, and source.

Setup Instructions

1. **Install Dependencies**: Ensure you have Python installed along with the `requests` library. You can install the library using pip:

```
```bash
```

```
pip install requests
```

...

2. **Locate LLM**: Make sure you have a locally-hosted LLM running at the specified ``OLLAMA_URL``.
3. **Configure Constants**: Update the constants in the script (``OLLAMA_URL``, ``MODEL_NAME``, ``VALID_EFFECTS``, ``PIECE_NAMES``, ``FALLBACK_EFFECTS``) to match your setup and preferences.
4. **Run the Script**: You can test the script by calling the ``get_narration`` function with sample move data.

### ***Future Improvements***

- **Enhanced Fallbacks**: Improve the fallback narrations and effects to be more varied and contextually relevant.
- **User Customization**: Allow users to customize the mappings of chess pieces to fantasy RPG units and the list of valid effects.
- **Model Training**: Train the LLM specifically for generating chess move narrations